

# Implementing 8-bit Computer on a FPGA

---

## **Design Lab Project**

Under the supervision of **Prof. Ritwik Kumar Layek**

Presented by **Group 5**

**Kaustubh Rakhade**

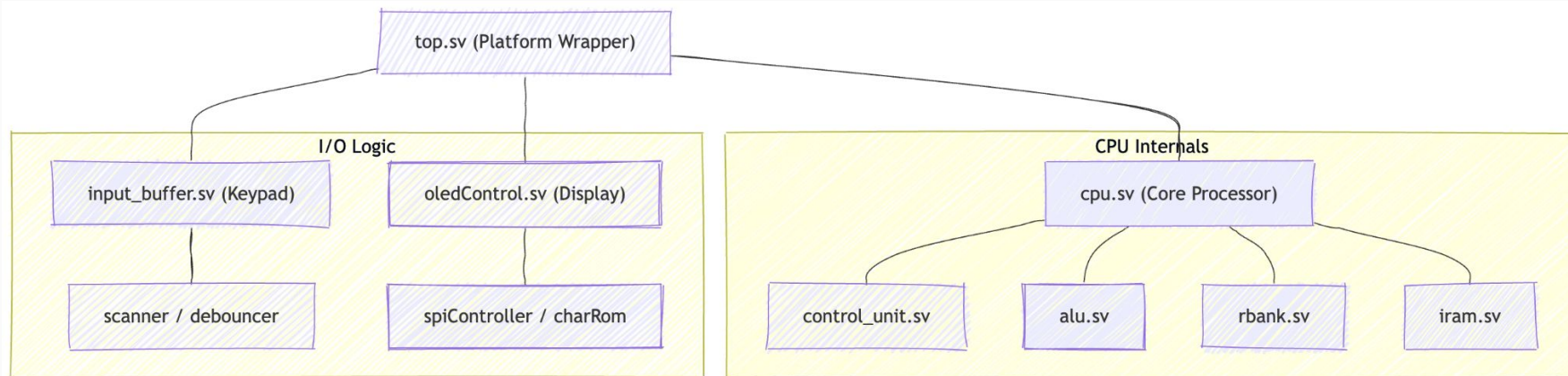
**Nangineni Chinmai Sabarish**

**Mamidi Surya Teja**

**Mayukh Mukherjee**

# Overview

A custom 8-bit Computer written in **SystemVerilog**, designed and synthesized using **AMD Vivado**, implemented and tested on **Zedboard FPGA**. This utilizes on board 100MHz clock, 4x4 Hexpad for user input and on board OLED screen for real time output. Programs are written in Hex, saved in a 256 byte instruction RAM.



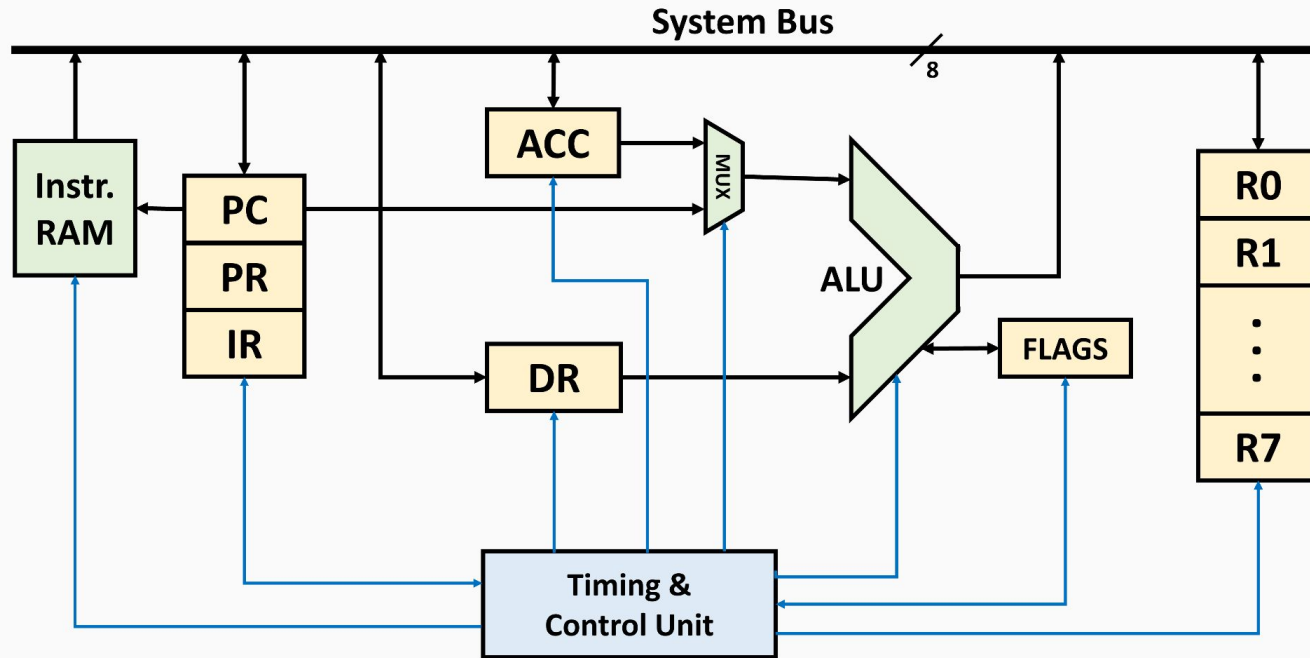
## Summary of ISA

- **Instruction Format:** 8-bit fixed-length opcodes, implemented using a finite state modelling.
  - [7:4] Operation Code (e.g., ADD, MOV, JMP).
  - [2:0] Register Index (R0-R7).
- Supports both Register and Immediate (#data) modes. Immediate mode takes a 2-byte instruction (opcode + data byte)
- Accumulator based architecture, i.e, most arithmetic and logic operations implicitly use the Accumulator (A) as the primary operand and destination.

### Instruction Groups

1. Arithmetic/Logic: ADD, SUBB, AND, OR, XOR.
  2. Unary/Shift: INC, DEC, CML (Complement), SL, SR.
  3. Data Movement: MOV (Reg  $\leftrightarrow$  A, Data  $\rightarrow$  Reg/A), PRINT A.
  4. Flow Control: JMP, JC (Carry), JNC, JZ (Zero), JNZ, CMP.
  5. System/Special: WAIT, HALT, PAUSE, INPUT.
-

## System Architecture



## Implementation of Instructions

The Control Unit reads the current instruction from the Instruction register, and generates and times control signals to various components. Here is an example for the ADD A, Rx operation:

```
T0:   iram_re = 1;           // IR ← IRAM[PC]; PC++
      ir_we = 1;
      pc_inc = 1;

T1:   dr_we = 1;             // DR ← Rx
      rb_re[REG_INDEX] = 1;

T2:   alu_op = ALU_ADD;      // A ← ALU(A, DR); T ← T0
      a_we = 1;
      alu_re = 1;
      reset_state = 1;
```

---

# Memory Subsystem

## **Synchronous Bus Architecture**

- A single 8-bit Tri-state Bus acts as the central data line.
- This is a "single-writer" system where the Control Unit ensures only one module drives the bus at a time.

## **Memory**

- Only IRAM, memory to store instructions exists.
- Program data is pre-loaded during bitstream generation using Xilinx Vivado .mem files (Block RAM initialization).

## **I/O Features**

- This system is based on Programmed Blocking I/O.
  - Traditional computers use interrupts (asynchronous hardware-to-CPU signals). Here, the CPU explicitly "requests" input and waits (stalls), allowing for simpler FPGA timing and deterministic execution.
-

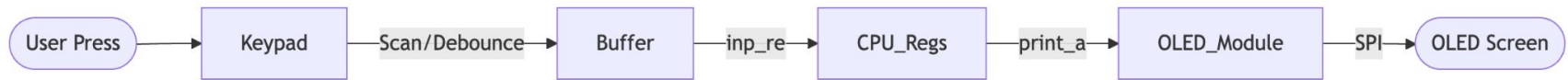
# Hexpad & Display Integration

## Hexpad Logic

- The FPGA drives each of the 4 columns HIGH one-by-one in a fast loop and reads the 4 row inputs.
- A button press completes the circuit between a specific Column and Row.
- Example: Column 3 (Active) + Row 3 (Detected) = Hex Key '9'.
- Debouncing: Mechanical noise is filtered by requiring a stable signal for 20ms before triggering

## Display Logic

- A special Print Register is added for communication between the CPU and display logic.
- When PRINT A is called, the Control Unit moves data from A to the Print Register via the bus.
- An ASCII converter & character ROM translates the value into a 6x8 pixel bitmap for the OLED screen.



# Thank you!

---

Any Questions?